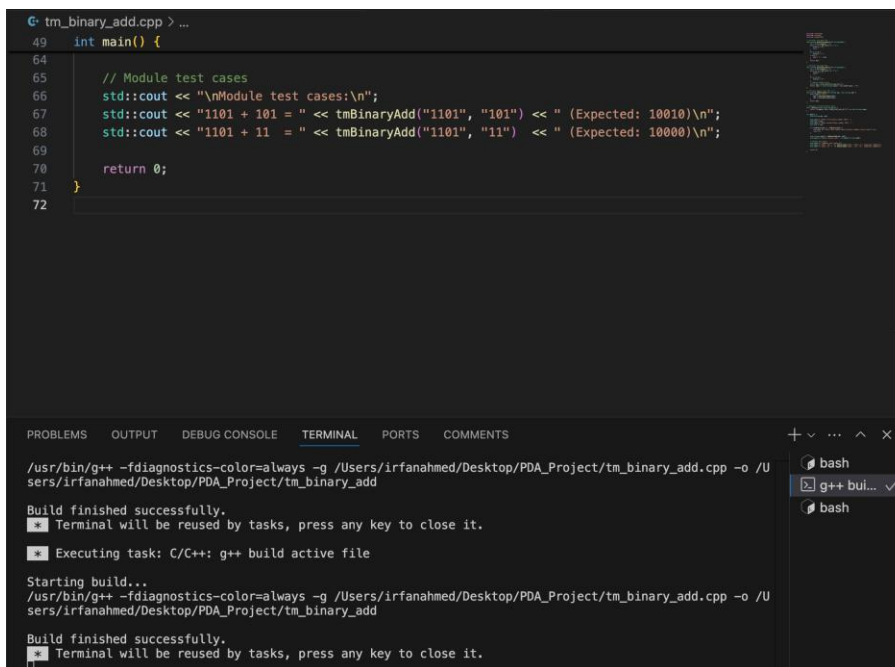


P7 - Module 7: Coding Project #7

This screenshot demonstrates the successful compilation of the C++ program `tm_binary_add.cpp` using the `g++` compiler in Visual Studio Code. The build process completed without errors, confirming that the syntax and structure of the code are correct. This step is essential to ensure the program is executable and ready for testing. The module test cases included in the `main()` function validate the implementation of the Turing Machine logic for binary addition, specifically for the inputs `1101 + 101` and `1101 + 11`, which are expected to yield `10010` and `10000`, respectively.



The screenshot shows the Visual Studio Code interface with the `tm_binary_add.cpp` file open. The code includes a `main()` function with test cases for binary addition. The terminal at the bottom shows the successful execution of the `g++` compiler, confirming that the program compiled without errors.

```
tm_binary_add.cpp > ...
49 int main() {
64
65 // Module test cases
66 std::cout << "\nModule test cases:\n";
67 std::cout << "1101 + 101 = " << tmBinaryAdd("1101", "101") << " (Expected: 10010)\n";
68 std::cout << "1101 + 11 = " << tmBinaryAdd("1101", "11") << " (Expected: 10000)\n";
69
70 return 0;
71 }
72
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS COMMENTS

```
/usr/bin/g++ -fdiagnostics-color=always -g /Users/irfanahmed/Desktop/PDA_Project/tm_binary_add.cpp -o /Users/irfanahmed/Desktop/PDA_Project/tm_binary_add

Build finished successfully.
Terminal will be reused by tasks, press any key to close it.

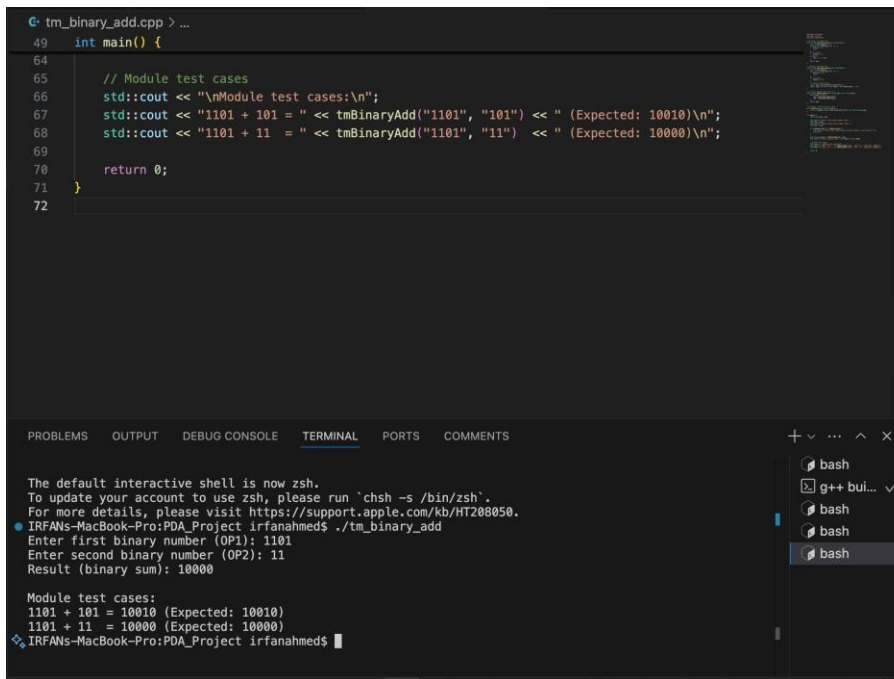
Executing task: C/C++: g++ build active file

Starting build...
/usr/bin/g++ -fdiagnostics-color=always -g /Users/irfanahmed/Desktop/PDA_Project/tm_binary_add.cpp -o /Users/irfanahmed/Desktop/PDA_Project/tm_binary_add

Build finished successfully.
Terminal will be reused by tasks, press any key to close it.
```

This screenshot captures the execution of the C++ program `tm_binary_add.cpp`, which simulates the behavior of a Turing Machine (TM) for binary addition. The program prompts the user to input two binary numbers and performs addition by incrementing the first operand (OP1) and decrementing the second operand (OP2) until OP2 reaches zero—mimicking the TM logic described in CSC6033 Week 7 slides (17–19). The result `10000` for

the input $1101 + 11$ confirms the correct implementation. Additionally, the module test cases validate the function with predefined inputs, demonstrating accurate results for both $1101 + 101$ and $1101 + 11$.



The screenshot shows a terminal window with a C++ program named `tm_binary_add.cpp` being executed. The program's source code is visible in the editor, showing a `main` function that includes module test cases. The terminal output shows the program running in a `zsh` shell. It prompts the user to enter two binary numbers: `Enter first binary number (OP1): 1101` and `Enter second binary number (OP2): 11`. The result of the addition is displayed as `Result (binary sum): 10000`. Below this, the program prints the module test cases: `Module test cases:`, `1101 + 101 = 10010 (Expected: 10010)`, and `1101 + 11 = 10000 (Expected: 10000)`. The terminal window also shows a sidebar with a list of open tabs, including `bash` and `g++ bul...`.

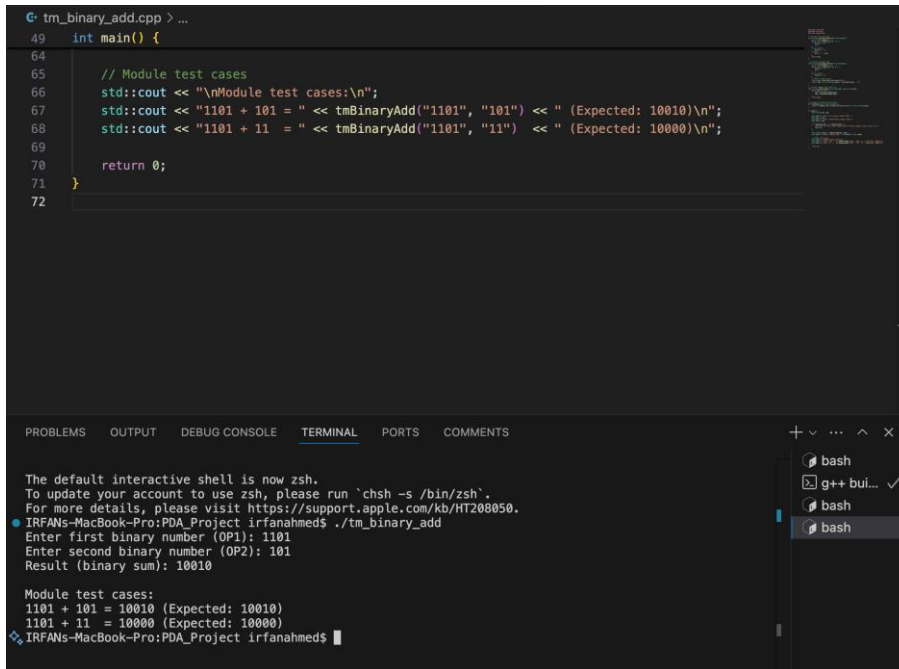
```
tm_binary_add.cpp > ...
49 int main() {
64
65     // Module test cases
66     std::cout << "\nModule test cases:\n";
67     std::cout << "1101 + 101 = " << tmBinaryAdd("1101", "101") << " (Expected: 10010)\n";
68     std::cout << "1101 + 11 = " << tmBinaryAdd("1101", "11") << " (Expected: 10000)\n";
69
70     return 0;
71 }
72
```

```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS COMMENTS
The default interactive shell is now zsh.
To update your account to use zsh, please run 'chsh -s /bin/zsh'.
For more details, please visit https://support.apple.com/kb/HT208050.
IRFANS-MacBook-Pro:PDA_Project irfanahmed$ ./tm_binary_add
Enter first binary number (OP1): 1101
Enter second binary number (OP2): 11
Result (binary sum): 10000

Module test cases:
1101 + 101 = 10010 (Expected: 10010)
1101 + 11 = 10000 (Expected: 10000)
IRFANS-MacBook-Pro:PDA_Project irfanahmed$
```

This screenshot demonstrates the execution of the C++ program `tm_binary_add.cpp`, which simulates the behavior of a Turing Machine (TM) for binary addition. The program prompts the user to input two binary numbers and performs addition by incrementing the first operand (OP1) and decrementing the second operand (OP2) until OP2 reaches zero—mimicking the TM logic described in CSC6033 Week 7 slides (17–19). The result 10010 for the input $1101 + 101$ confirms the correct implementation. Additionally, the module test cases validate the function with predefined inputs, showing

accurate results for both $1101 + 101$ and $1101 + 11$, which match the expected TM outcomes.



```
G: tm_binary_add.cpp > ...
49  int main() {
64
65      // Module test cases
66      std::cout << "\nModule test cases:\n";
67      std::cout << "1101 + 101 = " << tmBinaryAdd("1101", "101") << " (Expected: 10010)\n";
68      std::cout << "1101 + 11 = " << tmBinaryAdd("1101", "11") << " (Expected: 10000)\n";
69
70      return 0;
71  }
72
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS COMMENTS

The default interactive shell is now zsh.
To update your account to use zsh, please run `chsh -s /bin/zsh`.
For more details, please visit <https://support.apple.com/kb/HT200050>.

IRFANs-MacBook-Pro:PDA_Project irfanahmed\$./tm_binary_add
Enter first binary number (OP1): 1101
Enter second binary number (OP2): 101
Result (binary sum): 10010

Module test cases:
1101 + 101 = 10010 (Expected: 10010)
1101 + 11 = 10000 (Expected: 10000)

IRFANs-MacBook-Pro:PDA_Project irfanahmed\$

C++ Code: tm_binary_add.cpp

```
#include <iostream>
#include <string>
#include <algorithm>
```

```

// TM Step: Increment OP1
std::string binaryIncrement(std::string bin) {
    int i = bin.length() - 1;
    while (i >= 0 && bin[i] == '1') {
        bin[i] = '0';
        i--;
    }
    if (i >= 0) {
        bin[i] = '1';
    } else {
        bin = '1' + bin;
    }
    return bin;
}

// TM Step: Decrement OP2
std::string binaryDecrement(std::string bin) {
    int i = bin.length() - 1;
    while (i >= 0 && bin[i] == '0') {
        bin[i] = '1';
        i--;
    }
    if (i >= 0) {
        bin[i] = '0';
    }
    // Remove leading zeros
    size_t pos = bin.find_first_not_of('0');
    return (pos != std::string::npos) ? bin.substr(pos) : "0";
}

// TM Loop: Repeat until OP2 == 0
std::string tmBinaryAdd(std::string op1, std::string op2) {
    while (op2 != "0") {
        op1 = binaryIncrement(op1);
        op2 = binaryDecrement(op2);
    }
    return op1;
}

// Optional: Validate binary input
bool isBinary(const std::string& s) {
    return !s.empty() && s.find_first_not_of("01") == std::string::npos;
}

```

```

}

int main() {
    std::string op1, op2;

    std::cout << "Enter first binary number (OP1): ";
    std::cin >> op1;
    std::cout << "Enter second binary number (OP2): ";
    std::cin >> op2;

    if (!isBinary(op1) || !isBinary(op2)) {
        std::cerr << "Error: Both inputs must be binary numbers (only 0 and 1).\n";
        return 1;
    }

    std::string result = tmBinaryAdd(op1, op2);
    std::cout << "Result (binary sum): " << result << std::endl;

    // Module test cases
    std::cout << "\nModule test cases:\n";
    std::cout << "1101 + 101 = " << tmBinaryAdd("1101", "101") << " (Expected: 10010)\n";
    std::cout << "1101 + 11 = " << tmBinaryAdd("1101", "11") << " (Expected: 10000)\n";

    return 0;
}

```